Chapter 3

The Modern R and Bioconductor MS Ecosystem

Lucas VHH TRAN[1]

Abstract

High-throughput mass spectrometry produces gigabytes of spectral time-series and high-dimensional feature arrays that exceed the computational capacity of standard spreadsheets and interactive graphical interfaces. The Bioconductor project provides a formal, peer-reviewed software ecosystem designed for high-throughput biological assays. Bioconductor establishes two architectural standards for computational mass spectrometry: formal S4 classes that enforce structural coordinate invariants between assay matrices and biological metadata, and modular storage backends that separate user-facing operations from on-disk memory representation. Operating on raw spectra via pointer-based backends (`MsBackendMzR`*) prevents random-access memory exhaustion when processing hundreds of gigabytes of raw data. At the quantification tier, containers such as* `SummarizedExperiment` *and* `QFeatures` *maintain relational integrity across hierarchical transitions from peptide-spectrum matches (PSMs) to proteins. This chapter presents the computational mechanics of vectorization, formal S4 object architectures, pluggable spectral backends, tidyverse data transformations on real PSM tables, and empirical Bayes variance moderation in R 4.6.1.*

Keywords: Bioconductor, S4 formal classes, Spectra backends, SummarizedExperiment, QFeatures, Vectorization, Empirical Bayes shrinkage, Reproducible pipelines.

Chapter Overview and Architectural Roadmap

Table 3.1 summarizes the foundational architecture, software containers, and computational mechanics of the modern R and Bioconductor mass spectrometry ecosystem.

Methodological Note

Tutor Warning: Package Installation Protocol

Installing Bioconductor packages with base R's `install.packages()` bypasses Bioconductor release coordination. Bioconductor packages share interdependent S4 class definitions tied to specific Biocon-

[1]CK-CARE Davos, Christine Kühne--Center for Allergy Research and Education, Davos, Switzerland. Email: `lucas.tran@ck-care.ch`

Table 3.1: Architectural components and data structures in the Bioconductor MS ecosystem.

Computational Layer	Core Package / Container	Structural Guarantee	Analytical Function
1. Raw Spectra	`Spectra` (`MsBackendMzR`)	$O(1)$ indexed disk seek; lazy-load peak arrays	Peak picking, filtering, precursor extraction
2. Experiment Metadata	`MsExperiment`	Synchronized link between spectra files and sample annotations	Multi-file cohort management
3. Feature Quantitation	`SummarizedExperiment`	Exact coordinate invariance: $\dim(\text{assay})_1 = \dim(\text{rowData})_1$	2D metabolite/protein abundance matrices
4. Multi-level Omics	`QFeatures`	Relational directed acyclic graph linking PSMs to proteins	Hierarchical protein summarization
5. Statistical Inference	`limma`	Empirical Bayes variance shrinkage (d_0, s_0^2)	Small-sample differential abundance

ductor release versions. Using `install.packages()` pulls uncoordinated versions from CRAN mirrors, producing class mismatch errors such as `'undefined class "Spectra"'`. Always install Bioconductor packages using `BiocManager::install()` to ensure repository-level dependency reconciliation.

3.1 The R Mindset: Vectorization and SIMD Acceleration

R is an interpreted vector-oriented language built on compiled C and Fortran linear algebra subroutines. In base R, scalar for-loops evaluate bytecodes and verify memory pointers on each iteration. When an explicit loop increments an array element in interpreted R:

$$y_i = \log_2(x_i + 1), \quad i \in \{1, 2, \ldots, N\} \tag{3.1}$$

the interpreter verifies memory allocation and type consistency N times. For $N = 200{,}000$ MS intensity measurements, scalar loops produce substantial interpreter latency.

Vectorized operations dispatch contiguous memory buffers directly to underlying C subroutines using Single Instruction, Multiple Data (SIMD) processor instructions. Vectorized execution operates in compiled memory blocks without bytecode evaluation overhead:

$$\mathbf{y} = \log_2(\mathbf{x} + \mathbf{1}) \tag{3.2}$$

Listing 3.1 benchmarks scalar loop evaluation against vectorized transformation on 200,000 simulated Orbitrap intensity measurements.

R Lab Listing 1: Listing 3.1: Vectorization vs Explicit Loop Performance Benchmark [R 4.4 / Bioconductor]

```
set.seed(42)
n_vals <- 200000
raw_intensities <- rlnorm(n_vals, meanlog = 14, sdlog = 1.5)

# Benchmark 1: Explicit scalar loop
t_start_loop <- Sys.time()
log_loop <- numeric(n_vals)
for (i in seq_along(raw_intensities)) {
  log_loop[i] <- log2(raw_intensities[i] + 1)
}
t_loop <- as.numeric(difftime(Sys.time(), t_start_loop, units = "secs"))

# Benchmark 2: Vectorized log2 dispatch
t_start_vec <- Sys.time()
log_vec <- log2(raw_intensities + 1)
t_vec <- as.numeric(difftime(Sys.time(), t_start_vec, units = "secs"))

cat(sprintf("Vector Size: %d intensity measurements\n", n_vals))
cat(sprintf("  Explicit For-Loop: %.4f seconds\n", t_loop))
cat(sprintf("  Vectorized log2:   %.4f seconds\n", t_vec))
cat(sprintf("  Acceleration Factor: %.1fx faster\n", t_loop / t_vec))
cat(sprintf("  Numeric Identity Check: %s\n", all.equal(log_loop, log_vec)))
```

```
Vector Size: 200000 intensity measurements
Explicit For-Loop: 0.0681 seconds
Vectorized log2:   0.0057 seconds
Acceleration Factor: 12.0x faster
Numeric Identity Check: TRUE
```

The vectorized execution completes in 0.0057 seconds compared to 0.0681 seconds for the explicit scalar loop, achieving a 12.0× acceleration while maintaining numeric identity.

3.2 Formal S4 Object Architecture and Coordinate Invariants

Bioconductor relies on the S4 object-oriented system rather than the informal S3 system common in base R packages. An S3 object is an arbitrary list with an assigned class attribute; S3 does not validate whether mandatory components exist or whether data types match.

In contrast, S4 classes enforce rigid contracts through formal slot definitions and class validation methods. An S4 class requires explicit slot types (e.g., `numeric`, `character`, `DataFrame`). When an S4 object is instantiated or modified, validity methods confirm that dimensional and coordinate invariants hold.

In mass spectrometry quantification, the central data container is `SummarizedExperiment`. It maintains three coordinated components:

1. **Assays**: A list of 2D matrices $\mathbf{Y} \in \mathbb{R}^{p \times n}$, where p represents biological features (metabolites, peptides, or proteins) and n represents analytical samples.

2. **rowData**: A `DataFrame` of length p containing feature annotations (e.g., m/z, retention time, gene symbols).

3. **colData**: A `DataFrame` of length n containing sample phenotypic annotations (e.g., treatment group, batch, injection order).

The S4 validity constructor enforces coordinate invariance:

$$\text{nrow}(\mathbf{Y}) = \text{nrow}(\text{rowData}) = p, \quad \text{ncol}(\mathbf{Y}) = \text{nrow}(\text{colData}) = n \tag{3.3}$$

Subsetting a `SummarizedExperiment` by rows or columns automatically subsets the assay matrix, `rowData`, and `colData` simultaneously:

$$\text{se}[1{:}k,\ 1{:}m] \implies \mathbf{Y}_{k\times m}, \quad \text{rowData}_{k\times c_1}, \quad \text{colData}_{m\times c_2} \tag{3.4}$$

Listing 3.2 tests S4 validity checking when dimensional invariants are violated.

R Lab Listing 2: Listing 3.2: S4 Object Architecture and Validity Enforcement [R 4.4 / Bioconductor]

```
library(SummarizedExperiment)

# Construct valid SummarizedExperiment (20 features x 6 samples)
n_feat <- 20
n_samp <- 6
toy_assay <- matrix(rlnorm(n_feat * n_samp, 10, 1), nrow = n_feat, ncol = n_samp,
                    dimnames = list(paste0("Feat_", 1:n_feat), paste0("S_", 1:n_samp)))
row_ann <- DataFrame(gene = paste0("Gene_", 1:n_feat), row.names = paste0("Feat_", 1:n_feat))
col_ann <- DataFrame(group = rep(c("Ctrl", "Treat"), each = 3), row.names = paste0("S_", 1:n_samp))

valid_se <- SummarizedExperiment(assays = list(exprs = toy_assay),
                                 rowData = row_ann, colData = col_ann)

cat(sprintf("Valid SummarizedExperiment Class: %s\n", class(valid_se)))
cat(sprintf("Dimensions: %d rows x %d columns\n", nrow(valid_se), ncol(valid_se)))
cat(sprintf("Slots Present: %s\n", paste(slotNames(valid_se), collapse = ", ")))

# Test invalid construction: 15 rowData entries for a 20-row assay matrix
tryCatch({
  invalid_se <- SummarizedExperiment(
    assays  = list(exprs = toy_assay),
    rowData = DataFrame(gene = paste0("Gene_", 1:15))
  )
}, error = function(e) {
  cat(sprintf("Expected S4 Validity Catch:\n  %s\n", conditionMessage(e)))
})
```

```
Valid SummarizedExperiment Class: SummarizedExperiment
Dimensions: 20 rows x 6 columns
Slots Present: colData, assays, NAMES, elementMetadata, metadata
Expected S4 Validity Catch:
invalid class "SummarizedExperiment" object: 1: 'x@assays' is not parallel to 'x'
```

The S4 constructor halts execution when `rowData` has 15 rows while the assay matrix has 20 rows. This validity check prevents silent misalignment between feature annotations and measurement values.

3.3 Tidyverse Data Grammar on Real PSM Tables

Tidyverse packages (`dplyr`, `tidyr`, `ggplot2`) provide a consistent tabular grammar for mass spectrometry metadata and identification tables. Identification tables generated from search engines (such as Mascot,

Comet, MS-GF+, or MaxQuant) are imported into R as tabular structures. The native pipe operator (|>) chains operations sequentially:

$$\mathbf{T}_{\text{final}} = \mathbf{T}_0 \,|\!>\, f_1(\dots) \,|\!>\, f_2(\dots) \,|\!>\, f_3(\dots) \tag{3.5}$$

Five primary verbs manipulate identification tables:

- `filter()`: Selects subset rows based on logical conditions (e.g., target-decoy status, identification rank).
- `select()`: Extracts or drops specific columns.
- `mutate()`: Calculates transformed variables (e.g., logarithmic scores, mass measurement errors).
- `group_by() + summarise()`: Computes group-level summary metrics.
- `arrange()`: Sorts rows by specified variables.

Listing 3.3 applies this grammar to an `mzIdentML` identification file containing 5,802 raw PSMs from an Erwinia carotovora dataset in the `msdata` package.

R Lab Listing 3: Listing 3.3: Tidyverse Data Transformation on Real PSM Identifications [R 4.4 / Bioconductor]

```
library(PSMatch)
library(dplyr)
library(msdata)

# Load raw mzIdentML identification table
id_file <- system.file("ident",
  "TMT_Erwinia_1uLSike_Top10HCD_isol2_45stepped_60min_01-20141210.mzid",
  package = "msdata")

psms_raw <- PSMatch::PSM(id_file)
psms_df  <- as.data.frame(psms_raw)

# Tidyverse pipeline: Rank 1 target peptides grouped by charge state
psm_summary <- psms_df %>%
  filter(rank == 1, isDecoy == FALSE) %>%
  group_by(chargeState) %>%
  summarise(
    n_psms       = n(),
    n_unique_pep = n_distinct(sequence),
    mean_score   = round(mean(calculatedMassToCharge), 2),
    .groups      = "drop"
  ) %>%
  arrange(desc(n_psms))

cat(sprintf("Raw PSM Table: %d rows x %d columns\n", nrow(psms_df), ncol(psms_df)))
cat("Target PSM Charge State Distribution:\n")
print(as.data.frame(psm_summary))
```

```
Raw PSM Table: 5802 rows x 35 columns
Target PSM Charge State Distribution:
chargeState n_psms n_unique_pep mean_score
1           2  1568       1330    672.24
2           3   973        916    694.21
```

```
3          4   190        184    743.20
4          5    17         14    738.10
5          6     3          3    524.84
```

Of the 5,802 PSMs, rank-1 target identifications are concentrated in charge states $z = 2$ (1,568 PSMs, 1,330 unique peptides) and $z = 3$ (973 PSMs, 916 unique peptides). Higher charge states ($z = 4, 5, 6$) account for 210 PSMs.

3.4 Pluggable Backends and the Four Core Containers

Computational MS workflows process two distinct types of data: raw spectral time-series and processed feature matrices. Bioconductor addresses this through four core containers.

3.4.1 Pluggable Backends in Spectra

The `Spectra` container separates user operations (e.g., peak filtering, precursor isolation) from the underlying storage mechanism. Four storage backends provide different trade-offs:

1. `MsBackendMzR`: On-disk pointer backend. Raw m/z and intensity arrays remain inside `.mzML` files. An in-memory index table holds scan headers (0.5 MB per run), enabling $O(1)$ random-access seeking without loading full peak arrays into RAM.

2. `MsBackendDataFrame`: In-memory backend. Stores all peak arrays as R `DataFrame` objects. This backend is fast for small datasets ($< 1{,}000$ spectra) but consumes substantial memory on high-resolution instruments.

3. `MsBackendHdf5Peaks`: High-performance on-disk backend. Compresses peak arrays into hierarchical data format (HDF5) files, facilitating fast parallel reads across multi-core computing clusters.

4. `MsBackendSql`: Relational database backend. Offloads spectral data storage to a SQL database server, supporting multi-user collaborative environments.

3.4.2 The Four Core Data Containers

Table 3.2: Comparison of the four primary Bioconductor mass spectrometry containers.

Container	Data Domain	Key Characteristics	Creation Function
`Spectra`	Raw mass spectra	Virtualized time-series; pluggable backends	`Spectra()`
`MsExperiment`	Experimental cohort	Couples `Spectra` to sample annotations and LC-MS files	`readMsExperiment()`
`SummarizedExperiment`	2D Feature matrix	Enforces parallel coordinate alignment between rows/columns	`SummarizedExperiment()`
`QFeatures`	Hierarchical omics	Multi-tier assay graph linking PSM $\rightarrow$ peptide $\rightarrow$ protein	`QFeatures()`

3.5 Empirical Bayes Variance Moderation in Small-Sample MS

Differential abundance testing in mass spectrometry evaluates whether feature intensities shift significantly across biological conditions. For feature g across n samples, the linear model is:

$$\mathbf{y}_g = \mathbf{X}\beta_g + \varepsilon_g, \quad \varepsilon_g \sim \mathcal{N}(\mathbf{0}, \sigma_g^2 \mathbf{I}) \tag{3.6}$$

where $\mathbf{X}$ is the design matrix and σ_g^2 is the feature-specific residual variance.

In clinical and biological MS experiments, sample sizes are often small ($n = 3$ to 5 replicates per condition), yielding residual degrees of freedom $d_g \leq 4$. The sample variance s_g^2 has large sampling variability. When a feature has a spuriously small sample variance by chance, its standard error approaches zero:

$$t_g = \frac{\hat{\beta}_g}{\mathrm{se}(\hat{\beta}_g)} = \frac{\hat{\beta}_g}{s_g \sqrt{v_g}} \tag{3.7}$$

resulting in inflated t-statistics and false positives.

The `limma` empirical Bayes framework addresses this by modeling feature variances as random variables drawn from a scaled inverse chi-square prior distribution:

$$\frac{1}{\sigma_g^2} \sim \frac{1}{d_0 s_0^2} \chi^2_{d_0} \tag{3.8}$$

where s_0^2 represents the global prior variance and d_0 represents the prior degrees of freedom estimated from the ensemble of all measured features. The moderated variance $\tilde{s}_g^2$ shrinks the sample variance toward the global prior:

$$\tilde{s}_g^2 = \frac{d_0 s_0^2 + d_g s_g^2}{d_0 + d_g} \tag{3.9}$$

The moderated t-statistic replaces s_g with $\tilde{s}_g$:

$$\tilde{t}_g = \frac{\hat{\beta}_g}{\tilde{s}_g \sqrt{v_g}} \tag{3.10}$$

Followed by Benjamini-Hochberg false discovery rate (FDR) control:

$$P_{\mathrm{adj}(i)} = \min_{k \geq i} \left(\frac{m \cdot P_{(k)}}{k} \right) \tag{3.11}$$

Listing 3.4 executes an end-to-end differential pipeline using `SummarizedExperiment` and `limma` on 50 features across 6 samples ($n = 3$ Control vs $n = 3$ Treated). Five features (METAB_001 to METAB_005) contain true $+2.0$ $\log_2$ fold-change spikes.

R Lab Listing 4: Listing 3.4: Minimal Bioconductor Differential Abundance Pipeline [R 4.4 / Bioconductor]

```
library(SummarizedExperiment)
library(limma)

set.seed(123)
n_features <- 50
n_samples  <- 6

base_means <- rlnorm(n_features, meanlog = 12, sdlog = 1.0)
```

```
exprs_mat  <- matrix(NA_real_, nrow = n_features, ncol = n_samples,
                     dimnames = list(paste0("METAB_", sprintf("%03d", 1:n_features)),
                                     paste0("Sample_", 1:n_samples)))

for (i in 1:n_features) {
  ctrl_vals  <- rnorm(3, mean = log2(base_means[i]), sd = 0.25)
  treat_mean <- if (i <= 5) log2(base_means[i]) + 2.0 else log2(base_means[i])
  treat_vals <- rnorm(3, mean = treat_mean, sd = 0.25)
  exprs_mat[i, ] <- c(ctrl_vals, treat_vals)
}

col_data <- DataFrame(
  condition = rep(c("Control", "Treated"), each = 3),
  row.names = colnames(exprs_mat)
)

se <- SummarizedExperiment(assays = list(log2_intensity = exprs_mat),
                           colData = col_data)

# Differential abundance modeling via limma
design <- model.matrix(~ condition, data = as.data.frame(colData(se)))
fit <- lmFit(assay(se), design)
fit <- eBayes(fit)
top_res <- topTable(fit, coef = "conditionTreated", number = 10)

print(round(top_res[, c("logFC", "AveExpr", "t", "P.Value", "adj.P.Val")], 4))
cat(sprintf("\nTrue Positive Recovery (FDR <= 0.05): %d / 5 (%.0f%%)\n",
            sum(top_res$adj.P.Val[1:5] <= 0.05),
            sum(top_res$adj.P.Val[1:5] <= 0.05) / 5 * 100))
cat(sprintf("False Positive Rate in Unchanged Features: %.2f%%\n",
            sum(top_res$adj.P.Val[-(1:5)] <= 0.05) / (n_features - 5) * 100))
```

```
            logFC AveExpr      t P.Value adj.P.Val
METAB_003  2.2690 20.4936 13.6511  0.0000   0.0000
METAB_001  2.2065 17.6221 12.5596  0.0000   0.0000
METAB_002  2.0778 17.9491 11.4209  0.0000   0.0000
METAB_005  1.8974 18.4520 10.5241  0.0000   0.0000
METAB_004  1.6255 18.4336  7.6173  0.0000   0.0000
METAB_036  0.5087 18.4799  3.0233  0.0069   0.0574
METAB_029  0.4609 15.6429  2.7691  0.0121   0.0756
METAB_044 -0.5122 20.5666 -2.7684  0.0121   0.0756
METAB_031 -0.3925 17.9047 -2.1728  0.0424   0.2356
METAB_048 -0.3993 16.5373 -2.1076  0.0483   0.2416
True Positive Recovery (FDR <= 0.05): 5 / 5 (100%)
False Positive Rate in Unchanged Features: 0.00%
```

The empirical Bayes pipeline recovers all 5 spiked features at FDR ≤ 0.05 (100% sensitivity). The estimated $\log_2$ fold-changes range from $+1.6255$ to $+2.2690$. The false positive rate among unchanged features is 0.00% (the top non-spiked feature, METAB_036, has adj.P.Val $= 0.0574$).

3.6 Reproducibility Infrastructure

Reproducible computational MS workflows require explicit dependency management and build automation:

1. **Package Version Locking (`renv`)**: Records exact versions of CRAN and Bioconductor packages in a version-controlled `renv.lock` manifest.

2. **Target Automation (`targets`)**: Implements dependency graph tracking. Peak picking and chromatographic alignment steps are executed once and cached on disk. Subsequent runs recompute only targets whose code or inputs changed.

3. **Literate Documentation (Quarto)**: Combines explanatory text, mathematical notation, and executable code chunks into reproducible PDF and HTML reports.

3.7 Worked Exercises

Exercise 3.1: Core Bioconductor Container Architectures

Problem Statement: Describe the four primary Bioconductor containers used in computational mass spectrometry: `Spectra`, `MsExperiment`, `SummarizedExperiment`, and `QFeatures`. State the mathematical data structure of each container and specify the Bioconductor constructor function.
Solution:

1. **`Spectra`**: Represents raw spectral time-series $(m/z, I, t_{\text{ret}}, \text{MS}_{\text{level}})$. Created via `Spectra::Spectra(file, backend = MsBackendMzR())`. Peak arrays are lazy-loaded on demand.

2. **`MsExperiment`**: Couples multi-file spectral streams to experimental design metadata. Created via `MsExperiment::readMsExperiment(spectraFiles, sampleData)`.

3. **`SummarizedExperiment`**: Represents a 2D quantification matrix $\mathbf{Y} \in \mathbb{R}^{p \times n}$ with row annotations $\mathbf{R} \in \mathbb{R}^{p \times k_1}$ and column annotations $\mathbf{C} \in \mathbb{R}^{n \times k_2}$. Instantiated using constructor `SummarizedExperiment( assays, rowData, colData)`.

4. **`QFeatures`**: Represents a hierarchical directed acyclic graph linking PSM-, peptide-, and protein-level `SummarizedExperiment` assays. Created via `QFeatures::QFeatures()` or `QFeatures::readQFeatures()`.

Exercise 3.2: Vectorized vs For-Loop Execution Dynamics

Problem Statement: In a laboratory benchmark processing 200,000 intensity values, a scalar for-loop required 0.0681 seconds, whereas vectorized dispatch required 0.0057 seconds (12.0× acceleration). Explain the underlying computer science mechanism responsible for this performance difference.
Solution: In base R, a scalar for-loop evaluates R interpreter bytecodes on every iteration, resolving memory addresses, checking object types, and allocating intermediate scalar variables. The total execution time scales as $O(N)$ with substantial per-iteration interpreter overhead.

In contrast, the vectorized function `log2(x + 1)` dispatches a contiguous memory buffer of 64-bit IEEE floating-point numbers directly to compiled C code. The compiled loop executes SIMD instructions on the CPU register without entering the R evaluator loop. Memory is preallocated in a single block, avoiding repeated memory reallocation.

Exercise 3.3: Tidyverse Translation of Base R Subsetting

Problem Statement: Translate the following base R filtering operation into an idiomatic tidyverse pipeline:

```
tmp1 <- psm_table[psm_table$msLevel == 2, ]
tmp2 <- tmp1[tmp1$xcorr >= 2.5, ]
tmp3 <- tmp2[order(tmp2$xcorr, decreasing = TRUE), ]
result <- tmp3[1:10, c("sequence", "xcorr", "protein_id")]
```

Solution: The operation filters for MS2 spectra with XCorr ≥ 2.5, sorts in descending order, and extracts the top 10 rows. The tidyverse translation is:

```
result <- psm_table %>%
  filter(msLevel == 2, xcorr >= 2.5) %>%
  arrange(desc(xcorr)) %>%
  slice_head(n = 10) %>%
  select(sequence, xcorr, protein_id)
```

This avoids creating intermediate variables `tmp1`, `tmp2`, and `tmp3`.

Exercise 3.4: Empirical Bayes Moderation in Small-Sample MS

Problem Statement: In an LC-MS experiment with $n = 3$ replicates per group ($d_g = 4$ degrees of freedom), explain why raw t-tests produce high false discovery rates and demonstrate how empirical Bayes moderation in `limma` stabilizes feature variance estimates.

Solution: With $d_g = 4$, the sample variance s_g^2 has large sampling variance ($\mathrm{Var}(s_g^2) = 2\sigma_g^4/d_g$). Features that exhibit low sample variance by chance yield small standard error denominators ($s_g\sqrt{v_g} \approx 0$). This inflates raw t-statistics, producing false positives.

Empirical Bayes moderation estimates a global prior variance s_0^2 and prior degrees of freedom d_0 across all features. The moderated variance shrinks feature variances toward the global prior:

$$\tilde{s}_g^2 = \frac{d_0 s_0^2 + d_g s_g^2}{d_0 + d_g} \tag{3.12}$$

When s_g^2 is spuriously close to zero, $\tilde{s}_g^2$ is bounded below by $\frac{d_0}{d_0+d_g} s_0^2$, preventing false-positive test statistics caused by tiny denominators.

Bibliography

[1] Gentleman, R.C., Carey, V.J., Bates, D.M., et al. Bioconductor: open software development for computational biology and bioinformatics. *Genome Biology*, 5(10):R80, 2004. https://doi.org/10.1186/gb-2004-5-10-r80.

[2] Gatto, L., Rainer, J. *msdata: Various Mass Spectrometry Raw Data and Example Files*. R package version 0.44.0, Bioconductor, 2024. https://doi.org/10.18129/B9.bioc.msdata.

[3] Rainer, J., Vicini, A., Salzer, L., Stanstrup, J., Badia, J.M., Neumann, S., Stravs, M.A., Verri Hernandes, V., Gatto, L. A modular and expandable ecosystem for metabolomics data annotation in R. *Metabolites*, 12(2):173, 2022. https://doi.org/10.3390/metabo12020173.

[4] Smyth, G.K. Linear models and empirical bayes methods for assessing differential expression in microarray experiments. *Statistical Applications in Genetics and Molecular Biology*, 3(1):Article 3, 2004. https://doi.org/10.2202/1544-6115.1027.

[5] Wickham, H., Averick, M., Bryan, J., et al. Welcome to the tidyverse. *Journal of Open Source Software*, 4(43):1686, 2019. https://doi.org/10.21105/joss.01686.

[6] Huber, W., Carey, V.J., Gentleman, R., et al. Orchestrating high-throughput genomic analysis with Bioconductor. *Nature Methods*, 12(2):115--121, 2015. https://doi.org/10.1038/nmeth.3252.